CS 1050 Technical Documentation

Click table of contents below to refresh after adding more headings

Overview

When you add information in your own words demonstrating your understanding. Do not use exact words from lectures or images from lectures.

DOs and DON'Ts

DO: Provide clear and simple explanations in your own words (be concise)
- Use short sentences, small paragraphs
- Use lists and tables to organize information
- Use standard industry terms but explain in simple language
- Use AI Tools to help you learn but use lectures first

DON'T copy my explanations from the slides as yours.

DO Use Code Snippets and Visuals to reduce lengthy explanations:

- Include examples you explored or created
- Include comments to describe parts of code
- Include Screenshots such as from your debugger
- Include your own drawings such as for memory, arrays, etc.
- Include images you have AI create (cite that AI created)
- Include images from the Internet with the link

DON'T use my slides as your image and explanations

DO Include Resources that give steps or help explain concepts

- You can include resources given in the lecture or find your own but
  - Include website link with summary
  - Include links to videos with summary of content

DON't use AI to create your technical documentation.

General Resources

- Your first source should always be your course lecture slides and documents or links to information in the lectures
  - F26_CS1050_Student_Course_Resources
  - Java Environment Setup
  - Introduction to Programming with Java - use the sections shared with you to practice ideas from the lecture.
- Next resource is to see Deb or learning assistance during office hours
- If you're unsure whether a source is trustworthy, ask your instructor first.

  ⚠️ Avoid These Sources
  - TPointTech https://www.tpointtech.com/
  - W3Schools – Java, syntax, data types, etc.: https://www.w3schools.com/java/
  - Sites without a known author or date
  - AI-generated blogs that do not cite documentation
  - Use of AI should be last as it will probably give you information you haven't learned or understand. It is expected you will be documenting concepts from class and book using your code examples.

Other Tools you should use:

- Eclipse
- Visualize Java Code - used to step through code and edit code

SD01 Software Development and Tools

Version Control

- Use version control management tools (Git/GitHub) to manage and document development with GitHub Desktop.

IDE and Debugging

- Write and debug programs in IDE (Eclipse)

Analyze requirements

Analyze requirements(user stories and acceptance criteria) to break problems into smaller testable tasks

Design and Testing Plan

- Apply Test-Driven Development (TDD): Create unit test cases, design algorithms in pseudocode, implement code (structured, readable and documented), debug and unit test incrementally
- Develop a documented plan by creating unit test cases and algorithm designs
- Use UML to design and implement multiple class programs.

Implement Quality Documented Code

- Apply Java naming conventions for variables, constants, methods, and classes.
- Write comments at top of file and inline comments to document code.
- Use constants and variables and avoid hard coding literals
- Document method code following javadoc commenting
- Use Single Responsibility Principle (SRP) and Do Not Repeat Yourself (DRY) to implement modular, readable and testable solutions

Module 1: Programming Fundamentals and Tools

Add the information to demonstrate the following objectives and organize with headings:

- Declaring, initializing, and assigning variables.

- Compare constants and variables
- Order of operations, integer division and the modulus operator.
- Memory allocation based on data type and potential overflow errors during program execution
- Select appropriate numeric primitive data types (int, double) based on the type of data and required precision.
- Apply assignment operators and basic arithmetic operators to form valid expressions.
- Implicit and explicit conversion rules and casting
- Mixed number (doubles and integers) operations
- Standard output using System class and print methods.
- Identify different types of Errors
    - Syntax error. Include a screenshot of your IDE showing there is a syntax error
    - Runtime error. Include a screenshot of your IDE showing
    - Logical errors.

Heading 2 Example

Heading 3 Example

Module 2: Predefined Classes, Methods, and Decision Structures

Add the information to demonstrate the following objectives and organize with headings:

- Using built-in classes (System, Scanner, String, Character, Math) and their methods; parameters and return values
- Distinguish between characters (char) and Strings.
- String as an array of characters and use String class methods.
- Use Boolean values and relational operators to compare data.
- Distinguish between if-else, multi-way if, and nested if statements to control program flow.
- Scope of variables
- Use logical operators (&&, ||, !) to combine conditions.
- Use of Switch statements for multi-way selection

Heading 2 Example

Heading 3 Example

Module 3: Loops and Methods

Add the information to demonstrate the following objectives and organize with headings:

- Compare different types of loops (for, while, do-while).
- Use sentinel values and boolean flags to control loop execution.
- Distinguish scope of variables in control structures and methods.
- Use loops and nested control structures to control program flow.
- Write and call methods to pass and return values.
- Trace memory during method calls to evaluate stack frame memory and pass-by-value.

Heading 2 Example

Heading 3 Example

Module 4: Arrays

Add the information to demonstrate the following objectives and organize with headings:

- Declaring, allocating, initializing 1D arrays.
- Reference variables, array memory, and out-of-bounds issues.
- Passing arrays by reference to and from methods
- Trace memory during method calls to distinguish reference variables vs. primitive variables, stack vs. heap memory allocation.
- Design algorithms for processing arrays.
- Designing methods that operate on arrays using loops.

Heading 2 Example

Heading 3 Example

Module 5: Classes and Objects

Add the information to demonstrate the following objectives and organize with headings:

- Use classes to define blueprints for creating objects that combine data (fields) and behaviors (methods).
- Define and implement constructors, including default, parameterized, and overloaded constructors, to initialize objects in different ways
- Encapsulation using access modifiers and getters/setters to protect and manage private data.
- Reference variables for objects, stack/heap diagrams.
- Arrays of objects
- Simple file input/output
- Design Implement and Test Java classes that model real-world problems

Heading 2 Example

Heading 3 Example